Lesson 10 Teacher's Key: Play It Again

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Extracting existing _ready() code into a named, reusable function (start_game()) - the single clearest, most explicit version of the 'factor out the reset routine' idea this week, since Godot's function syntax makes the refactor completely visible.

Level 1 - Full Worked Solution

- start_game() lives on GameManager (which owns the shared state) and resets it, then asks Main's spawn_enemies() to rebuild the wave - the spawn loop is the old _ready() body, moved into a function.
- _ready() shrinks to a single line: start_game.call_deferred(). This is the clearest illustration all week of 'the entry point calls the reusable routine, rather than containing the logic directly.'
- get_tree().get_nodes_in_group("enemies") is the mechanism that makes clearing old enemies possible - without the Group assignment in Setup, there'd be no reliable way to find every existing enemy to remove.

Highlighted lines are new this week; everything else is code you already have.

```
# GameManager's script (complete):
var enemies_remaining := 5
var lives := 3
var score := 0
var game_active := true

func _ready():
    start_game.call_deferred()

func start_game():
    for old_enemy in get_tree().get_nodes_in_group("enemies"):
        old_enemy.queue_free()
    enemies_remaining = 5
    lives = 3
    score = 0
    game_active = true
    get_node("../WinText").hide()
    get_node("../LoseText").hide()
    update_hud()
    get_parent().spawn_enemies()
```

```
func _input(event):
    if event is InputEventKey and event.pressed and event.keycode ==
KEY_R:
        if not game_active:
            start_game()

func update_hud():
    get_node("../ScoreText").text = "Score: " + str(score)
    get_node("../LivesText").text = "Lives: " + str(lives)

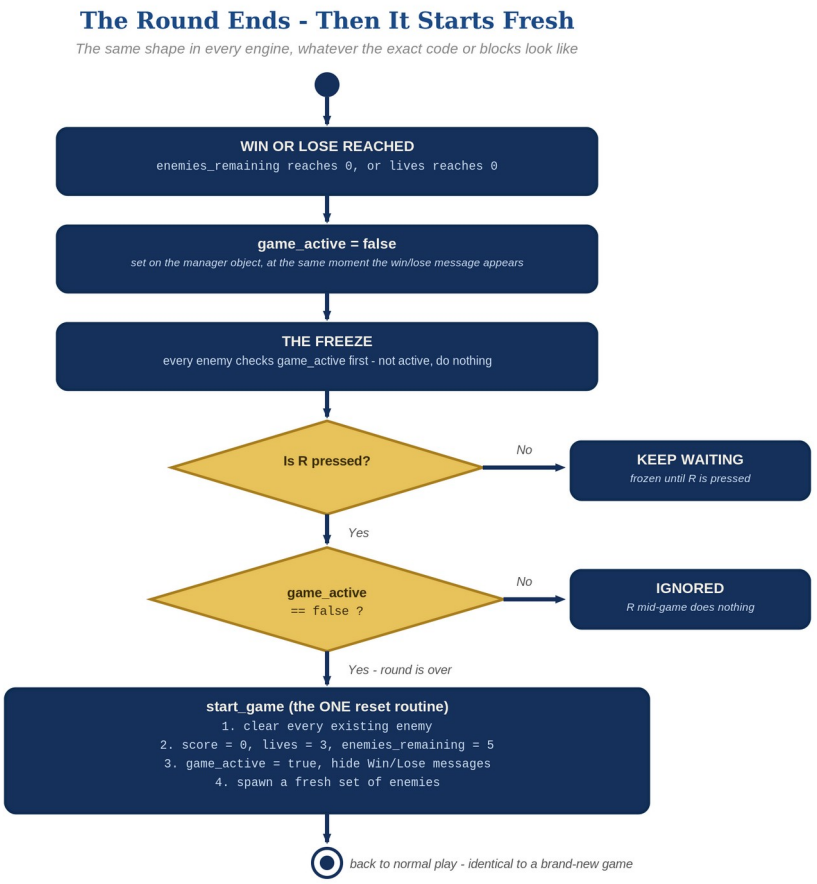
func on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
        game_active = false
        get_node("../WinText").text = "You Win! Press R to Restart"
        get_node("../WinText").show()

func on_enemy_escaped():
    lives -= 1
    update_hud()
    if lives <= 0:
        game_active = false
        get_node("../LoseText").text = "Game Over - Press R to Restart"
        get_node("../LoseText").show()

# Main scene's script (complete):
var enemy_scene := preload("res://enemy.tscn")

func spawn_enemies():
    for enemy_count in range(5):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(150 + enemy_count * 200, 50)
        add_child(new_enemy)
        new_enemy.destroyed.connect($GameManager.on_enemy_destroyed)
        new_enemy.escaped.connect($GameManager.on_enemy_escaped)

# enemy.tscn's script - add as the very first line of _process():
func _process(delta):
    if not get_node("../GameManager").game_active:
        return
    position.y += enemy_speed * delta
    if position.y > 700:
        escaped.emit()
        queue_free()
```



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

Why This Structure

- This mirrors exactly what Lesson 9 did with update_hud() (one function, called from multiple places) - start_game() is the same idea, applied to setup instead of display.
- _input(event) is used here (rather than an Input Map action) specifically to keep this lesson self-contained - no Project Settings configuration required, consistent with the course's self-containment principle.

Level 2 / Level 3 Talking Points

- Level 3 bug hunt: kills reduce enemies_remaining but escapes only reduce lives, so a round with some kills and some escapes (for example 3 and 2) ends with no enemies left, no win and no loss, and R does nothing because game_active is still true. This started in Lesson 6 (win = all killed) and Lesson 8 (lose = lives gone). A good fix, and the one shown here, is that an escape also resolves an enemy and the win check requires lives above 0. Accept any design that guarantees every round ends in exactly one outcome (for example a third 'Round Over' state, or a check that no enemies remain and none are in flight). Test it with 3 kills and 2 escapes, then 4 and 1, then 0 and 5.

- Ask students to name every place start_game() is called from (_ready(), and _input()) and confirm they understand why both call sites needed to exist before this lesson (one for the very first game, one for every subsequent one).
- For students comparing engines: Godot's start_game() function and GameMaker's scr_start_game Script resource are conceptually identical - both are 'a named block of reusable code,' just with different syntax.

Level 2 - complete code:

```
# Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
# Checks: after a win and after a loss, R restores 0/3/5 with no leftover
enemies; R during play does nothing.
```

Level 3 - code (additions highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
# Level 3 visual: fade the message in (GameManager's script).
func on_enemy_destroyed():
    # ...Level 1 body unchanged...
    if enemies_remaining <= 0:
        game_active = false
        get_node("../WinText").text = "You Win! Press R to Restart"
        get_node("../WinText").modulate.a = 0.0
        get_node("../WinText").show()
        create_tween().tween_property(get_node("../WinText"),
"modulate:a", 1.0, 0.5)

# BUG FIX (one good answer): an escape also resolves an enemy.
GameManager's script:
func on_enemy_escaped():
    lives -= 1
    enemies_remaining -= 1
    update_hud()
    if lives <= 0:
        game_active = false
        get_node("../LoseText").text = "Game Over - Press R to Restart"
        get_node("../LoseText").show()
    elif enemies_remaining <= 0:
        game_active = false
        get_node("../WinText").text = "You Win! Press R to Restart"
        get_node("../WinText").show()
```

Common Student Mistakes

- Calling start_game() directly from GameManager's _ready(): GameManager is a child of Main, its _ready() runs before Main has finished setting up, and Main refuses add_child() while busy, so no enemies appear and no error is shown in the game. The fix is start_game.call_deferred(). Worth a short explanation of ready order for students who hit it.
- Game-state drift: the student added start-up code or a game-over effect beyond the handout (extra green-flag scripts, an animation, a color change, a hidden object) and did not reset it on restart. Symptom: a restart that looks almost right. Walk through the Handout's "Audit Your Game State" list with them.
- Forgetting to add spawned enemies to the "enemies" Group, silently breaking the clear-on-restart step (they'll still exist and move, just never found by get_nodes_in_group()).

- Placing "if not game_active: return" anywhere other than the first line of _process() - enemies would still perform part of their per-frame logic before bailing out.
- Comparing event.keycode against a raw integer instead of the KEY_R constant - technically works if the integer happens to match, but fragile and easy to get wrong.